

Fundamentos

Introdução

Prof. Edson Alves

Faculdade UnB Gama

Sumário

1. **Motivação**
2. **Contexto Histórico**

Paradigmas de Programação

O que é paradigma?

Paradigmas de Programação

O que é programação?

Termos associados aos paradigmas de programação

Programação Imperativa

CÁLCULO λ

Programação Funcional

Combinadores

Orientação à objetos

Multiagentes

Programação Concorrente

PROGRAMAÇÃO DECLARATIVA

Orientação à Configuração

Programação Estruturada

Máquinas de Turing

Benefícios do estudo dos paradigmas de programação

- ▶ Aumento da capacidade de expressar ideias
- ▶ Escolhas bem fundamentadas das linguagens de programação a serem utilizadas em um projeto
- ▶ Melhora na capacidade de aprendizado de novas linguagens
- ▶ Uso mais eficaz das linguagens já conhecidas
- ▶ Maior entendimento das diferentes implementações de um mesmo conceito
- ▶ Visão mais ampla da computação como um todo

Os ideais de Leibniz

O matemático alemão Gottfried Wilhelm Leibniz (1646-1716) tinha dois ideais:

1. Criar uma “*linguagem universal*” na qual todos os problemas pudessem ser descritos
2. Encontrar um método de decisão para que todos os problemas descritos nesta linguagem pudessem ser resolvidos

Teoria dos conjuntos e lógica de primeira ordem

No que diz respeito aos problemas matemáticos, o primeiro ideal de Leibniz poderia ser alcançado, em teoria, por meio de uma Teoria de Conjuntos formulada em termos de uma Lógica de Primeira Ordem.

O matemático/lógico/filósofo inglês Bertrand Russell (1872-1970) e o matemático/lógico alemão Ernst Zermelo (1871-1953) trouxeram grandes contribuições para esta questão.

A grande questão

O segundo ideal trazia consigo uma grande questão filosófica, que ficou conhecida como *Entscheidungsproblem*:

É possível resolver todos os problemas descritos na linguagem universal?

Turing e Church

A resposta negativa para o *Entscheidungsproblem* foi dada em 1936, independentemente, por dois grandes matemáticos. Para tal, eles precisaram formalizar a noção de decidibilidade, ou computabilidade:

- ▶ Alonzo Church (1936) inventou um sistema formal, denominado Cálculo λ , e definiu a noção de função computável por meio deste sistema
- ▶ Alan Turing (1936/1937) criou as Máquinas de Turing, e definiu computabilidade em termos destas máquinas

Características de ambos modelos estiveram presentes nas diversas linguagens de programação ao longo da história.

IA e a grande questão

A linguagem natural poderia ser, de fato, a “linguagem universal”?

Um LLM seria um método de decisão para a solução de todos os problemas descritos em linguagem natural?

Problema simples + solução (Gemini)

quanto é a metade de dois mais dois?



Essa é uma pegadinha clássica! A resposta depende de como você interpreta a frase:

- **3 (A resposta gramatical/matemática):** Se você pegar "a metade de dois" (que é 1) e somar "mais dois", o resultado é 3.
- **2 (A resposta da soma primeiro):** Se você considerar "a metade de (dois mais dois)", primeiro você soma $2+2$ para obter 4, e a metade de quatro é 2.

Na maioria das vezes que fazem essa pergunta, a resposta esperada para a pegadinha é 3!

Problema simples + solução (Gemini)

e qual seria a resposta correta para este problema?

Matematicamente e gramaticalmente falando, a resposta correta é 3.

Isso acontece por causa da **ordem das operações matemáticas** (também conhecida como regra de precedência). Em uma expressão, a divisão ou multiplicação deve ser resolvida antes da adição ou subtração.

Portanto, a conta estruturada é:

$(1/2 \text{ de } 2) + 2$

$1 + 2 = 3$

Para que a resposta correta fosse 2, a frase precisaria ter uma pausa ou vírgula estruturando a ideia, como: "Quanto é a metade de: dois mais dois?" ou "Quanto é a metade da soma de dois mais dois?". Sem essa pausa, a matemática dita que a resposta é 3.

Referências

1. **BARENDREGT**, Henk; **BARENSEN**, Erik. *Introduction to Lambda Calculus*, March 2000.
2. **HALE**, M. *Essentials of Mathematics: Introduction to Theory, Proof, and the Professional Culture*, Mathematical Association of America, 2003. (**eBrary**)
3. Wikipédia. [Alan Turing](#), acesso em 09/01/2020.
4. Wikipédia. [Alonzo Church](#), acesso em 09/01/2020.
5. Wikipédia. [Bertrand Russel](#), acesso em 09/01/2020.
6. Wikipédia. [Ernst Zermelo](#), acesso em 09/01/2020.
7. Wikipédia. [Gottfried Wilhelm Leibniz](#), acesso em 09/01/2020.